

EXPERIMENT – 01

Aim: Design and train a feedforward neural network using discrete perceptron algorithm

Theory:

The Perceptron is one of the simplest artificial neural network architectures, introduced by Frank Rosenblatt in 1957. It is primarily used for binary classification.

At that time, traditional methods like Statistical Machine Learning and Conventional Programming were commonly used for predictions. Despite being one of the simplest forms of artificial neural networks, the Perceptron model proved to be highly effective in solving specific classification problems, laying the groundwork for advancements in AI and machine learning.

Perceptron is a type of neural network that performs binary classification that maps input features to an output decision, usually classifying data into one of two categories, such as 0 or 1.

Perceptron consists of a single layer of input nodes that are fully connected to a layer of output nodes. It is particularly good at learning linearly separable patterns. It utilizes a variation of artificial neurons called Threshold Logic Units (TLU), which were first introduced by McCulloch and Walter Pitts in the 1940s. This foundational model has played a crucial role in the development of more advanced neural networks and machine learning algorithms.

Types of Perceptron

- Single-Layer Perceptron is a type of perceptron is limited to learning linearly separable patterns. It is effective for tasks where the data can be divided into distinct categories through a straight line. While powerful in its simplicity, it struggles with more complex problems where the relationship between inputs and outputs is non-linear.
- Multi-Layer Perceptron possess enhanced processing capabilities as they consist of two or more layers, adept at handling more complex patterns and relationships within the data.

Basic Components of Perceptron

A Perceptron is composed of key components that work together to process information and make predictions.

- Input Features: The perceptron takes multiple input features, each representing a characteristic of the input data.
- Weights: Each input feature is assigned a weight that determines its influence on the output. These weights are adjusted during training to find the optimal values.
- Summation Function: The perceptron calculates the weighted sum of its inputs, combining them with their respective weights.

- **Activation Function:** The weighted sum is passed through the Heaviside step function, comparing it to a threshold to produce a binary output (0 or 1).
- **Output:** The final output is determined by the activation function, often used for binary classification tasks.
- **Bias:** The bias term helps the perceptron make adjustments independent of the input, improving its flexibility in learning.
- **Learning Algorithm:** The perceptron adjusts its weights and bias using a learning algorithm, such as the Perceptron Learning Rule, to minimize prediction errors.

How does Perceptron work?

A weight is assigned to each input node of a perceptron, indicating the importance of that input in determining the output. The Perceptron's output is calculated as a weighted sum of the inputs, which is then passed through an activation function to decide whether the Perceptron will fire.

The weighted sum is computed as:

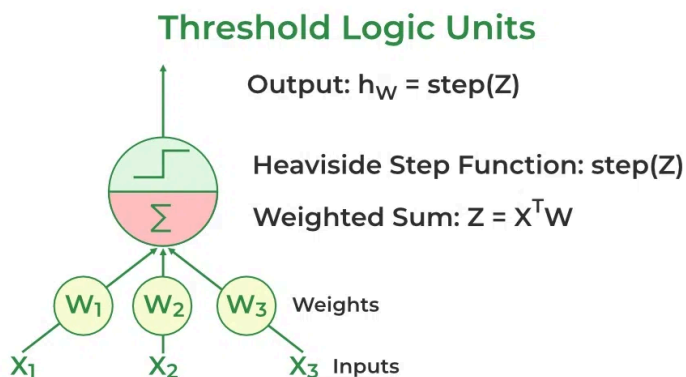
$$z = w_1x_1 + w_2x_2 + \dots + w_nx_n = X^T W$$

The step function compares this weighted sum to a threshold. If the input is larger than the threshold value, the output is 1; otherwise, it's 0. This is the most common activation function used in Perceptrons are represented by the Heaviside step function:

$$h(z) = \begin{cases} 0 & \text{if } z < \text{Threshold} \\ 1 & \text{if } z \geq \text{Threshold} \end{cases}$$

if $z < \text{Threshold}$

if $z \geq \text{Threshold}$



A perceptron consists of a single layer of Threshold Logic Units (TLU), with each TLU fully connected to all input nodes.

The **Discrete Perceptron** specifically operates in a thresholded manner, producing outputs in discrete values (typically $\{-1, 1\}$ or $\{0, 1\}$) rather than continuous values.

1. Model Representation

A perceptron is a type of **linear classifier**, meaning it makes predictions based on a linear function of the input features. The mathematical model consists of:

$$y = f(w \cdot x + b)$$

where:

- $x = [x_1, x_2, \dots, x_n]$ is the input feature vector.
- $w = [w_1, w_2, \dots, w_n]$ is the weight vector.
- b is the bias term.
- $w \cdot x = \sum_{i=1}^n w_i x_i$ is the weighted sum of inputs.
- $f(\cdot)$ is the **activation function**, typically a step function:

$$f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ -1, & \text{otherwise} \end{cases}$$

This means the perceptron outputs a discrete value **either 1 or -1** (or equivalently 1 or 0, depending on the convention).

2. Learning Algorithm (Training)

The perceptron learning rule is an **online learning** algorithm, meaning it updates weights after each misclassified example.

Initialization

- Initialize the weight vector w and bias b to small random values (e.g., 0 or small numbers).

Update Rule

For each training example (x, y) , where $y \in \{-1, 1\}$:

1. Compute the prediction: $\hat{y} = f(w \cdot x + b)$
2. Update weights **only if misclassified** ($\hat{y} \neq y$):

$$w \leftarrow w + \eta y x \quad b \leftarrow b + \eta y$$
where η (eta) is the **learning rate** (typically a small positive number).

Stopping Criteria

- The perceptron stops when **all training examples are correctly classified**.
- If the dataset is **not linearly separable**, the perceptron **never converges**.

Advantages

Simple and efficient – Easy to implement and works well on linearly separable data.

Guaranteed convergence – If the data is **linearly separable**, the algorithm will converge in a finite number of steps.

Limitations

Only works for linearly separable data – It fails to classify data that is not linearly separable (e.g., XOR problem).

Fixed decision boundary – Cannot model complex decision boundaries.

Sensitive to feature scaling – Performance can be affected by how input features are scaled.